

Universidad Mariano Gálvez de Guatemala

Centro UMG Guastatoya



Facultad de Ingeniería en Sistemas - Octavo Ciclo

Sección: A

Análisis de Sistemas II

Ing. Richard Ortiz

MEJORA DE DISEÑO VERIFICABLE DEL MÓDULO ASII-13

Signos vitales y alertas por valores anormales

Aplicación del Principio de Sustitución de Liskov (LSP)

Josué Fernando Hicho García

Carné: 1890-23-2862

GitHub: Jhos-hgnu

Dato de entrega	Valor
URL del repositorio	https://github.com/Jhos-hgnu/asii-2026-modulo-signos-vitales.git
Rama evaluada	feature/semana-02-solid
Commit o etiqueta evaluada	semana-02-lsp-v1

Semana 02 - Diseño SOLID

Índice

Sección	Página
1. Introducción	3
2. Contexto y alcance del módulo	3
3. Requisitos funcionales (RF)	3
4. Requisitos no funcionales (RNF)	4
5. Criterios de aceptación	4
6. Principio SOLID seleccionado: LSP	5
7. Diseño inicial - Antes de aplicar LSP	6
8. Diseño propuesto aplicando LSP	6
9. Justificación de responsabilidades y dependencias	7
10. Comparación antes vs. después	7
11. Validación del diseño y evidencia verificable	8
12. Matriz de trazabilidad	9
13. Evidencia Git y reproducibilidad	9
14. Limitaciones	11
15. Conclusión	12
16. Bibliografía	13
Anexo A. Checklist de entrega	14

1. Introducción

El módulo ASII-13 de Signos vitales y alertas por valores anormales gestiona el registro, consulta y evaluación de mediciones clínicas asociadas a pacientes. Durante la primera semana se documentaron los actores, casos de uso, flujo de registro, evaluación automática y generación de alertas. En esta segunda semana, la actividad exige convertir esos requisitos en una mejora de diseño verificable aplicando específicamente el Principio de Sustitución de Liskov (LSP).

La propuesta se concentra en el flujo asignado: captura de signos vitales, evaluación de umbrales y generación de alerta. El objetivo no es etiquetar el diseño como "SOLID", sino demostrar mediante contratos, diagramas antes/después y escenarios de sustitución que las reglas concretas de evaluación pueden utilizarse de forma intercambiable sin romper el comportamiento esperado del flujo.

Antecedente de la semana 01

El diagnóstico previo definió un proceso en el que el personal autorizado registra signos vitales, el sistema valida la información, consulta rangos de referencia, evalúa los valores y genera una alerta cuando detecta una condición anormal. También se identificaron requisitos de autenticación, permisos, aislamiento por tenant y trazabilidad.

2.2. Flujo asignado para la semana 02

- Capturar y registrar los signos vitales de un paciente.
- Evaluar las mediciones utilizando reglas o umbrales de referencia.
- Determinar si cada medición se encuentra en estado normal o anormal.
- Generar una alerta vinculada al registro cuando exista una condición anormal.
- Permitir que las reglas de evaluación evolucionen sin romper el flujo que las consume.

2.3. Fuera del alcance de esta mejora

La propuesta no implementa el sistema completo ni define valores clínicos reales. Tampoco sustituye la valoración médica. La semana 02 se limita a una decisión de diseño y a su validación documental.

3. Requisitos funcionales (RF)

ID	Requisito funcional
RF-01	El sistema debe permitir registrar signos vitales asociados a un paciente.
RF-02	El sistema debe validar los datos de las mediciones antes de almacenarlos.
RF-03	El sistema debe evaluar automáticamente las mediciones mediante reglas de umbral configuradas.
RF-04	El sistema debe identificar si los valores evaluados se encuentran dentro o fuera del rango esperado.
RF-05	El sistema debe generar una alerta cuando una o más reglas determinen un resultado anormal.

ID	Requisito funcional
RF-06	La alerta generada debe quedar vinculada al registro de signos vitales que originó la condición.
RF-07	El personal autorizado debe poder consultar el historial y las alertas relacionadas con el registro.

4. Requisitos no funcionales (RNF)

ID	Requisito no funcional
RNF-01	Las operaciones protegidas deben requerir autenticación y respetar los mecanismos de seguridad definidos por el proyecto.
RNF-02	El acceso a registro, consulta y revisión de alertas debe respetar roles y permisos.
RNF-03	Los registros deben mantenerse aislados según el tenant correspondiente.
RNF-04	Las operaciones relevantes deben conservar trazabilidad de usuario, fecha y acción.
RNF-05	Las reglas concretas de evaluación deben respetar un contrato común de entrada y salida para que puedan sustituirse sin modificar el componente que las consume.
RNF-06	Una regla concreta no debe fortalecer las precondiciones ni debilitar las postcondiciones definidas por el contrato de evaluación.
RNF-07	La incorporación de una regla de evaluación compatible no debe exigir cambios en MotorEvaluacionVital ni en el flujo de registro.

5. Criterios de aceptación

CA-01 - Registro: Dado un usuario autenticado y autorizado y un paciente existente, cuando ingrese mediciones válidas, entonces el sistema debe almacenar el registro correctamente.

CA-02 - Validación: Dado un conjunto de datos inválidos, cuando se intente registrar, entonces el sistema debe rechazar la operación e informar los errores correspondientes.

CA-03 - Evaluación: Dado un registro válido, cuando finalice el registro, entonces el sistema debe ejecutar las reglas de evaluación aplicables.

CA-04 - Resultado: Dada una regla de evaluación, cuando procese un VitalSign, entonces debe devolver un ResultadoEvaluacion conforme al contrato común.

CA-05 - Alerta: Dado que al menos una regla devuelva un resultado anormal, cuando finalice la evaluación, entonces el sistema debe generar una alerta vinculada al registro.

CA-06 - Sustitución LSP: Dadas dos implementaciones válidas de ReglaEvaluacionVital, cuando una sustituya a la otra dentro de MotorEvaluacionVital, entonces el motor debe seguir ejecutándose sin requerir cambios en su lógica de invocación.

CA-07 - Precondiciones: Dada cualquier regla concreta compatible, cuando el motor entregue un VitalSign válido, entonces la regla no debe exigir que el cliente conozca un tipo especial ni utilizar una firma distinta.

CA-08 - Postcondiciones: Dada cualquier regla concreta, cuando finalice evaluar(), entonces debe producir un ResultadoEvaluacion interpretable por el mismo flujo de alertas.

6. Principio SOLID seleccionado: LSP

6.1. Definición y fuente obligatoria

La fuente obligatoria de MVP Cluster presenta el Principio de Sustitución de Liskov indicando que “*una clase derivada de otra debe poder ser sustituida por su clase base*”. En términos del diseño propuesto, una implementación concreta de una regla de evaluación debe poder utilizarse donde el sistema espera la abstracción ReglaEvaluacionVital sin provocar comportamientos inesperados ni obligar al cliente a conocer el subtipo.

Fuente: MVP Cluster, “Diseño de software 2”, sección Liskov Substitution. Consulta indicada por la actividad: 31 de julio de 2026.

6.2. Contrato de sustitución para el módulo

Elemento	Definición
Abstracción	ReglaEvaluacionVital
Entrada común	VitalSign validado y perteneciente al contexto autorizado.
Operación	evaluar(registro: VitalSign)
Salida común	ResultadoEvaluacion.
Precondición	No exigir una forma de invocación más restrictiva que la definida por la abstracción.
Postcondición	Devolver siempre un resultado interpretable por MotorEvaluacionVital y GestorAlertas.
Efectos permitidos	Evaluar el registro y producir un resultado; no modificar el contrato del cliente.
Comportamiento no permitido	Obligar al motor a usar if/switch por subtipo, cambiar la firma del método, o lanzar excepciones únicamente porque la regla concreta requiere una llamada especial.

7. Diseño inicial - Antes de aplicar LSP

En el diseño inicial, el flujo intenta utilizar una abstracción ReglaUmbral cuya operación principal recibe un único valor decimal. Esto funciona para reglas simples como temperatura o frecuencia cardiaca, pero genera un problema cuando aparece una regla con necesidades distintas, por ejemplo presión arterial, que requiere más de un valor relacionado. Si el subtipo cambia la forma de invocación, fortalece la precondición o exige tratamiento especial, deja de ser sustituible por la abstracción esperada.

Diseño inicial - Riesgo de incumplimiento de LSP

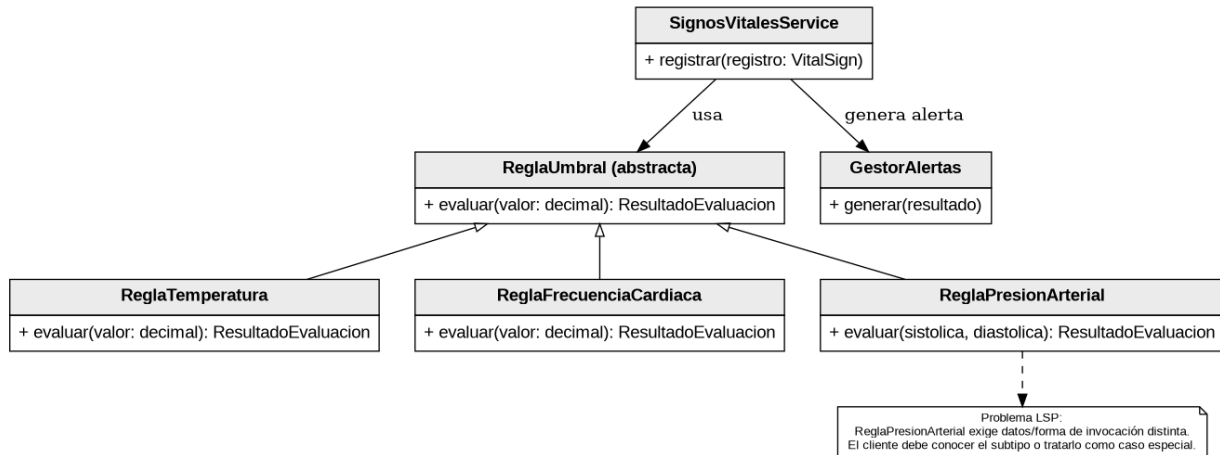


Figura 1. Diseño inicial: una regla concreta exige un tratamiento especial y rompe la sustitución esperada.

7.1. Problema verificable

Aspecto	Consecuencia
Cambio de regla	El cliente necesita conocer qué subtipo está utilizando.
Firma o precondition	Una regla concreta puede exigir parámetros adicionales.
Motor de evaluación	Termina incorporando condicionales por tipo de regla.
Efecto en el flujo	La sustitución de una regla puede romper el proceso captura -> evaluación -> alerta.
Resultado	No se puede afirmar que todos los subtipos sean intercambiables respecto de la abstracción.

8. Diseño propuesto aplicando LSP

El diseño propuesto cambia el contrato de evaluación para que todas las reglas concretas reciban el mismo objeto VitalSign y devuelvan el mismo tipo ResultadoEvaluacion. MotorEvaluacionVital trabaja únicamente contra la abstracción ReglaEvaluacionVital y recorre una colección de reglas compatibles. El servicio de registro no necesita conocer la clase concreta que evaluó el signo vital, y GestorAlertas recibe un resultado uniforme.

Diseño propuesto aplicando LSP

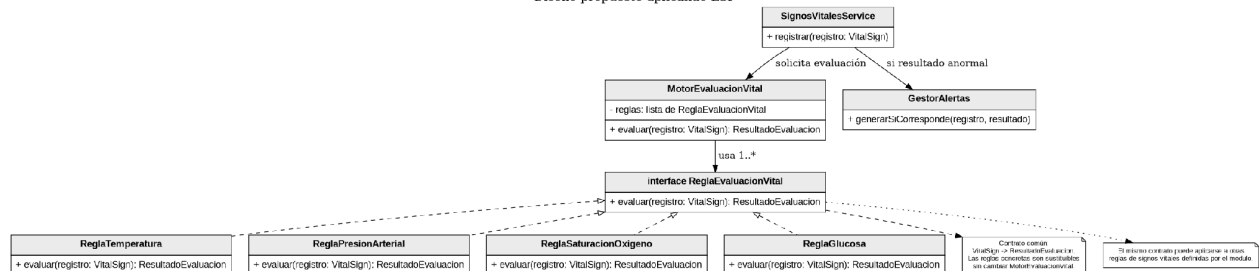


Figura 2. Diseño propuesto: las reglas concretas respetan un contrato común y pueden sustituirse dentro del motor.

8.1. Componentes y responsabilidades

Componente	Responsabilidad
SignosVitalesService	Coordina la captura y registro y solicita la evaluación del VitalSign.
MotorEvaluacionVital	Ejecuta las reglas configuradas sin depender de sus clases concretas.
ReglaEvaluacionVital	Define el contrato común evaluar(VitalSign): ResultadoEvaluacion.
Reglas concretas	Aplican la lógica específica de un signo vital respetando el mismo contrato.
ResultadoEvaluacion	Representa de forma uniforme el resultado normal/anormal y la información necesaria para el flujo.
GestorAlertas	Genera la alerta cuando el resultado de evaluación lo requiere y la vincula al registro.

9. Justificación de responsabilidades y dependencias

La aplicación de LSP se verifica en la relación entre ReglaEvaluacionVital y sus implementaciones. El motor depende del contrato, no de una regla específica. Por ello, cambiar de ReglaTemperatura a ReglaSaturacionOxigeno, o incorporar otra regla compatible, no modifica la forma en que MotorEvaluacionVital invoca evaluar(). La particularidad clínica queda encapsulada dentro de la implementación concreta.

Esta decisión evita que el cliente tenga que preguntar por el tipo real mediante condicionales y evita que un subtipo introduzca requisitos de uso adicionales. La generación de alertas permanece después de la evaluación y consume un resultado común, por lo que el flujo completo conserva una estructura estable aun cuando cambien las reglas concretas.

10. Comparación antes vs. después

Aspecto	Diseño inicial	Diseño propuesto
Contrato de evaluación	Puede variar por subtipo.	Único: VitalSign -> ResultadoEvaluacion.
Conocimiento del subtipo	El cliente puede necesitarlo.	El motor trabaja contra ReglaEvaluacionVital.
Precondiciones	Una regla puede exigir datos o firma especial.	Las reglas aceptan el mismo tipo de entrada.
Postcondiciones	La salida puede ser inconsistente.	Todas devuelven ResultadoEvaluacion.
Sustitución	Puede romper el flujo.	Una regla compatible puede reemplazar a otra.
Motor de evaluación	Tiende a incorporar if/switch por tipo.	No requiere condicionales por clase concreta.

Aspecto	Diseño inicial	Diseño propuesto
Impacto del cambio	Un subtipo especial obliga a modificar al cliente.	La particularidad queda dentro de la regla concreta.
Principio	LSP no garantizado.	LSP aplicado mediante contrato uniforme y sustitución verificable.

11. Validación del diseño y evidencia verificable

Como esta entrega corresponde a una mejora de diseño previa a la implementación, la evidencia principal es una validación estática y reproducible del contrato. No se afirma que existan pruebas unitarias ejecutadas si todavía no se ha implementado el código.

ID	Escenario	Resultado esperado	Evaluación
V-01	Sustituir ReglaTemperatura por ReglaSaturacionOxigeno en la colección del motor.	MotorEvaluacionVital mantiene la misma llamada evaluar(VitalSign) y recibe ResultadoEvaluacion.	Cumple LSP si no cambia el cliente.
V-02	Agregar una nueva regla compatible.	El motor la consume a través de ReglaEvaluacionVital sin condicional por tipo.	Cumple LSP.
V-03	Una regla exige una firma distinta.	El motor tendría que conocer el subtipo o cambiar la llamada.	No cumple LSP; diseño rechazado.
V-04	Una regla devuelve un tipo distinto.	GestorAlertas no podría procesar el resultado de forma uniforme.	No cumple LSP; diseño rechazado.
V-05	Una regla válida no aplica porque el campo no está presente.	Debe devolver un estado compatible (por ejemplo NO_APLICA) según el contrato, no obligar al cliente a cambiar de firma.	Mantiene sustitución.

11.1. Evidencia técnica editable esperada

- Archivo PUMML del diseño inicial (por ejemplo: docs/semana-02/diagramas/lsp-antes.puml).
- Archivo PUMML del diseño propuesto (por ejemplo: docs/semana-02/diagramas/lsp-despues.puml).
- Imagen exportada de cada diagrama para facilitar su lectura en el documento.
- Documento final DOCX o PDF.
- DECLARACION_IA.md y GUIA_DEFENSA.md en el repositorio.
- Evidencia Git que permita identificar el commit evaluado.

12. Matriz de trazabilidad

ID	Requisito	Elemento de diseño	Evidencia	Trazabilidad
RF-01	Registrar signos vitales	SignosVitalesService	Diseño después / flujo de captura	Registro coordinado antes de evaluar.
RF-02	Validar mediciones	Validación previa al registro	Criterio CA-02	Evita evaluar datos inválidos.
RF-03	Evaluar mediciones	MotorEvaluacionVital + ReglaEvaluacionVital	Figura 2	El motor aplica reglas mediante contrato común.
RF-04	Clasificar resultado	ResultadoEvaluacion	CA-04 / CA-08	La salida es uniforme para cualquier regla.
RF-05	Generar alerta	GestorAlertas	Figura 2 / CA-05	Se activa a partir de un resultado anormal.
RF-06	Vincular alerta	GestorAlertas + VitalSign	CA-05	La alerta conserva relación con el registro origen.
RNF-05	Sustitución de reglas	ReglaEvaluacionVital	CA-06	Una implementación válida reemplaza a otra sin cambiar el motor.
RNF-06	Contrato LSP	Precondiciones/post condiciones	Sección 6.2	Evita fortalecer precondiciones o debilitar salidas.
RNF-07	Extensibilidad compatible	MotorEvaluacionVital	V-02	Agregar una regla compatible no modifica la lógica del motor.

13. Evidencia Git y reproducibilidad

La consigna exige que la versión evaluada pueda identificarse de manera verificable. Por ello, la portada debe registrar la URL del repositorio, la rama y el commit o etiqueta evaluada, y el documento debe conservar evidencia del historial y del árbol de archivos.

Dato	Evidencia
Repositorio	https://github.com/Jhos-hgnu/asii-2026-modulo-signos-vitales.git
Rama	feature/semana-02-solid
Etiqueta, si se utiliza	semana-02-lsp-v1

13.1. Árbol sugerido para semana 02

```
docs/  
  semana-02/  
    diagramas/  
      lsp-antes.puml  
      lsp-despues.puml  
    imagenes/  
      lsp-antes.png  
      lsp-despues.png  
    ModuloSignosVitales-Semana02-LSP.docx  
    EVIDENCIA_GIT.md (opcional, pero recomendado)  
  DECLARACION_IA.md  
  GUIA_DEFENSA.md
```

14. Limitaciones

La propuesta corresponde a una etapa de diseño y no demuestra por sí sola el comportamiento de una implementación real. La validación completa de LSP deberá complementarse posteriormente con pruebas unitarias y de integración que ejecuten distintas implementaciones de ReglaEvaluacionVital contra el mismo conjunto de pruebas de contrato. Además, los rangos clínicos concretos no se fijan en este documento; deben provenir de reglas aprobadas para el proyecto y no de valores inventados para la tarea.

15. Conclusión

La semana 02 permitió transformar el flujo de captura de signos vitales, evaluación de umbrales y generación de alertas en una mejora de diseño verificable. La decisión más relevante fue definir un contrato común, `ReglaEvaluacionVital`, para que cada regla concreta procese un `VitalSign` y produzca un `ResultadoEvaluacion` sin imponer al cliente una forma especial de uso. Con ello, `MotorEvaluacionVital` puede trabajar con distintas implementaciones de manera uniforme y la sustitución de una regla compatible no requiere modificar la lógica del flujo, lo que constituye la aplicación concreta de LSP en esta propuesta.

La principal limitación es que el diseño todavía debe validarse mediante código y pruebas de contrato cuando avance la implementación. Sin embargo, los RF/RNF, criterios de aceptación, contrato de sustitución, diagramas antes/después, escenarios de validación, matriz de trazabilidad y evidencia Git propuesta permiten demostrar de forma verificable cómo la decisión de diseño responde a la consigna asignada.

16. Bibliografía

MVP Cluster. “Diseño de software 2”. <https://mvpcluster.com/disenode-software-2/> (consulta: 31 de julio de 2026).

Object Management Group. (2017). Unified Modeling Language (UML), versión 2.5.1.

PHP Documentation. “Supported Versions” y manual de PDO. <https://www.php.net/docs.php> (referencia técnica aplicable cuando corresponda a la etapa de implementación).

Anexo A. Checklist de entrega

- Portada con URL, rama y commit o etiqueta evaluada.
- Índice actualizado.
- RF/RNF y criterios de aceptación.
- Principio LSP citado y justificado con la fuente obligatoria.
- Diseño antes y diseño después.
- Fuentes editables .puml en el repositorio.
- Imágenes exportadas de los diagramas.
- Matriz de trazabilidad.
- Evidencia Git con historial, SHA y árbol de archivos.
- DECLARACION_IA.md actualizado.
- GUIA_DEFENSA.md disponible.
- Sin secretos ni datos clínicos reales.